



UNIVERSITY OF LEEDS

Formal Language & Finite Automata

Context-free grammars & Pushdown automata

Dr Noleen Koehler

University of Leeds

All reading is from the "Introduction to the theory of computation" -
Micheal Sipser

- Context-free languages - Pages 101 - 117 (essential)

Context-free languages

Definition (Context-free grammar (CFG))

A **context-free grammar** is a 4-tuple (V, Σ, R, S) , where

1. V is a finite set called the **variables (non-terminals)**,
2. Σ is a finite set, disjoint from V , called the **terminals**,
3. R is a finite set of **production rules**, with each production rule being a variable and a string of variables and terminals, and
4. $S \in V$ is the **start variable**.

Example

Consider the following CFG,

$$S \rightarrow A$$

$$A \rightarrow 0A1$$

$$A \rightarrow B$$

$$B \rightarrow \#$$

- What are the variables?
- What are the terminals?
- What is the language generated by the grammar?

If u , v and w are strings of variables and terminals, and $A \rightarrow w$ is a rule of the grammar, we say that uAv **yields** uwv , written $uAv \Rightarrow uwv$.

u **derives** v , written $u \Rightarrow^* v$, if either $u = v$ or if a sequence $u_1, u_2, \dots, u_k$ exists for $k \geq 0$ and

$$u \Rightarrow u_1 \Rightarrow u_2 \Rightarrow \dots \Rightarrow u_k \Rightarrow v.$$

The **language of a grammar**, denoted $L(G)$ (a slight corruption of notation), is $\{w \in \Sigma^* \mid S \Rightarrow^* w\}$.

Definition

A string is derived **ambiguously** in context-free grammar G if it has two or more different leftmost derivation. Grammar G is **ambiguous** if it generates some string ambiguously.

Example

$$S \rightarrow aB \mid ab$$

$$A \rightarrow AB \mid a$$

$$B \rightarrow Abb \mid b$$

Is the language ambiguous? Hint: consider the string 'ab'.

Chomsky Normal Form (CNF)

Definition (Chomsky normal form)

A context-free grammar is in **Chomsky normal form** if every rule is of the form

$$A \rightarrow BC, \text{ or}$$

$$A \rightarrow a$$

where a is any terminal and A , B , and C are any variables— except that B and C may not be the start variable. In addition, we permit the rule $S \rightarrow \epsilon$, where S is the start variable.

Chomsky Normal Form (CNF)

Theorem

Any context-free language can be generated by a context-free grammar in Chomsky normal form.

Proof.

First, we add a new start variable S_0 and the rule $S_0 \rightarrow S$, where S was the original start variable. This change guarantees that the start variable doesn't occur on the right hand side of a rule.



Chomsky Normal Form (CNF)

Proof.

Secondly, we handle the ϵ -rules. We remove an ϵ -rule $A \rightarrow \epsilon$, where A is not the start variable. For each occurrence of an A on the right hand side of a rule, we add a new rule with that occurrence deleted. We repeat these steps until all ϵ -rules have been eliminated that don't involve the start variable.



Chomsky Normal Form (CNF)

Proof.

Third, we handle all unit rules. We remove a unit rule $A \rightarrow B$ and introduce a new set of rules. Whenever $B \rightarrow u$ appears as a rule we replace it with $A \rightarrow u$. We repeat these steps until all unit productions have been removed. Finally, we convert all remaining rules into the proper form. We replace each rule $A \rightarrow u_1 u_2 \dots u_k$ where $k \geq 3$ and each u_i is a variable or terminal symbol with the rules $A \rightarrow u_1 A_1$, $A_1 \rightarrow u_2 A_2$, $A_2 \rightarrow u_3 A_3$, $\dots$, and $A_{k-2} \rightarrow u_{k-1} u_k$. We replace any terminal u_i from the previous construction with the variable U_i and introduce a new rule $U_i \rightarrow u_i$. $\square$

Definition (Pushdown automata)

A **Pushdown automata** is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$, where Q , Σ , Γ , and F are all finite sets, and

1. Q is the set of states,
2. Σ is the input alphabet,
3. Γ is the stack alphabet,
4. $\delta : Q \times \Sigma_{\epsilon} \times \Gamma_{\epsilon} \rightarrow \mathcal{P}(Q \times \Gamma_{\epsilon})$ is the transition function,
5. $q_0 \in Q$ is the start state, and
6. $F \subseteq Q$ is the set of accept states.

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ be a pushdown automata and let $w = w_1 \dots w_n$ be a string where $w_i \in \Sigma$. The automaton M **accepts** w if a sequence of states $r_0, \dots, r_n \in Q$ and a sequence of strings $s_0, \dots, s_n \in \Gamma^*$ exists such that:

1. $r_0 = q_0$ and $s_0 = \epsilon$;
2. for $0 \leq i < n$ we have $(r_{i+1}, b) \in \delta(r_i, w_{i+1}, a)$ where $s_i = at$ and $s_{i+1} = bt$ for some $a, b \in \Gamma_\epsilon$ and $t \in \Gamma^*$;
3. $r_n \in F$.